SmartSafe Master Unit

Complete Design and Code Explanation of the Power-Failure Detection Module

Microprocessors End-Term Project

July 2026

Abstract

This report documents the power-monitoring subsystem used in the SmartSafe Master unit. The module detects a simulated reduction of a monitored supply with the ATmega32 analog comparator, rejects short disturbances with a software stability filter, reports accepted power-state transitions to the main program, cooperates with two different sleep strategies, and correctly handles a failure already present at startup. The explanation follows the supplied `power.c` implementation and covers the external voltage divider, comparator polarity, register configuration, interrupt behavior, atomic sections, startup synchronization, normal inactivity sleep, power-failure standby, restoration handling, timing constants, and the interaction between the interrupt service routine, Timer2, and the main loop.

Contents

1 Purpose and Design Requirements

The power module has four responsibilities:

1. Monitor a separate adjustable voltage representing the Master supply.
2. Detect a persistent drop below a selected threshold rather than reacting to a short spike or comparator chatter.
3. Report one logical event for each accepted transition: normal-to-failed and failed-to-normal.
4. Cooperate with the Master power-management policy:
   - normal inactivity uses Power-down sleep and permits only INT0 to wake the processor;
   - a confirmed power failure uses a dedicated standby state in which the comparator remains available to detect restoration.

The module deliberately separates *raw hardware state* from *accepted software state*. The analog comparator may change immediately and may toggle several times around the threshold. The rest of the application sees a transition only after the state remains stable for the configured confirmation interval.

2 Hardware Arrangement

2.1 Comparator inputs

With `ACBG=1`, the internal bandgap reference is connected to the non-inverting comparator input, denoted V_+. The divided monitored voltage is connected to `PB3/AIN1`, the inverting input V_-. The polarity is therefore:

$$ACO = 1 \iff V_+ > V_- \iff V_{BG} > V_{AIN1}.$$

The firmware interprets `ACO=1` as a power failure.

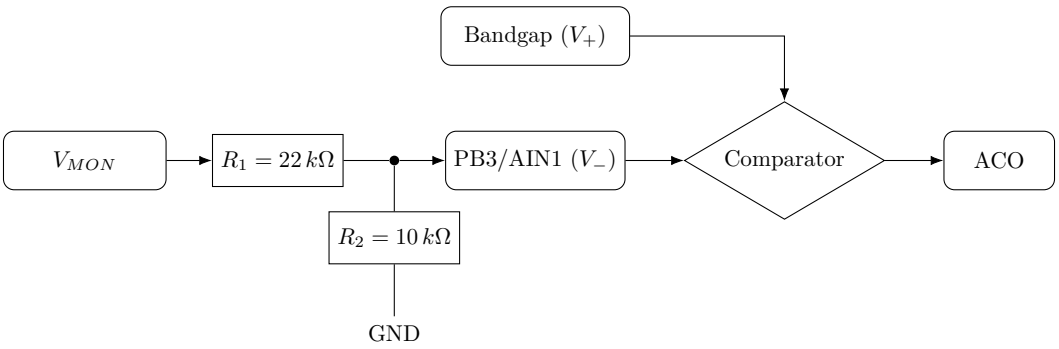

Figure 1: Power-monitoring voltage divider and comparator polarity.

2.2 Divider threshold

The divider output is

$$V_{AIN1} = V_{MON}\frac{R_2}{R_1 + R_2}.$$

For $R_1 = 22\,k\Omega$ and $R_2 = 10\,k\Omega$,

$$V_{AIN1} = 0.3125\,V_{MON}.$$

The comparator changes state when V_{AIN1} is approximately equal to the bandgap voltage. Using a nominal bandgap value of 1.23 V,

$$V_{MON,trip} = 1.23\frac{22+10}{10} \approx 3.94 \text{ V}.$$

Thus a nominal 5 V monitored source is treated as normal, while a reduction below roughly 4 V is treated as a failure. The exact point depends on the comparator model, bandgap tolerance, resistor tolerance, and simulation behavior.

3 Module State Variables

```
volatile uint8_t power_event_pending = 0;
volatile uint8_t power_fail_state = 0;

static volatile uint8_t power_candidate_state = 0;
static volatile uint8_t power_confirm_ticks = 0;
static volatile uint8_t power_confirmation_pending = 0;
```

Listing 1: Power-state variables.

Variable	Meaning
`power_fail_state`	Last accepted and software-filtered state. Zero means normal power; one means failed power. This is the authoritative state used by the application.
`power_event_pending`	Notification flag for `main.c`. It becomes one when a new accepted state must be processed.
`power_candidate_state`	Raw comparator state captured by the latest comparator transition.
`power_confirm_ticks`	Remaining 10 ms intervals before the candidate may be accepted.
`power_confirmation_pending`	Indicates that a candidate is currently being verified.

The accepted state and event flag are `volatile` because they are shared between the module and the application. The filter variables are also `volatile` because the comparator ISR and Timer2 ISR modify them while the main loop reads them.

4 Reading the Comparator Output

```
static uint8_t comparator_reports_power_fail(void)
{
    return (ACSR & (1 << ACO)) ? 1U : 0U;
}
```

Listing 2: Conversion of ACO into a logical failure state.

This helper isolates the hardware polarity from the rest of the code. A return value of one always means “power failed,” even though the actual hardware condition is “the bandgap input is greater than the divided monitored input.” This improves readability and avoids repeated direct interpretation of `ACO`.

5 Analog Comparator Register Configuration

The ATmega32 Analog Comparator Control and Status Register, `ACSR`, contains the fields shown below.

Table 2: ACSR fields relevant to the project.

Bit	Name	Project use
7	ACD	Comparator disable. Kept at zero so the comparator remains powered.
6	ACBG	Internal bandgap select. Set to one, connecting the bandgap to V_+.
5	ACO	Read-only comparator output. One is interpreted as failure.
4	ACI	Comparator interrupt flag. It is cleared by writing a logic one.
3	ACIE	Comparator interrupt enable. Set while transitions must generate interrupts.
2	ACIC	Timer1 input-capture connection. Kept at zero.
1:0	ACIS1:0	Interrupt mode. Both remain zero, selecting interrupt on comparator-output toggle.

The code intentionally uses complete assignments such as

```
ACSR = (1 << ACBG) | (1 << ACI);
ACSR = (1 << ACBG) | (1 << ACIE);
```

rather than read-modify-write operations. This matters because `ACI` is a write-one-to-clear flag. Writing zero leaves it unchanged; writing one clears it. An expression such as `ACSR &= ~(1<<ACI)` would not clear the flag.

5.1 Dedicated AIN1 rather than ADC multiplexer input

The comparator can optionally use the ADC multiplexer as its negative input when `ACME` is enabled under the required ADC conditions. This design uses the dedicated `AIN1/PB3` pin, so the code clears `ACME`:

```
#ifdef ACME
    SFIOR &= ~(1U << ACME);
#endif
```

This avoids accidental routing of an ADC channel into the comparator.

6 Initialization Sequence

```
void power_monitor_init(void)
{
    uint8_t initial_state;

    POWER_SENSE_DDR &= ~(1U << POWER_SENSE_PIN);
    POWER_SENSE_PORT &= ~(1U << POWER_SENSE_PIN);

#ifdef ACME
    SFIOR &= ~(1U << ACME);
```

```
#endif

    ACSR = (1U << ACBG);
    delay_ms(1);

    initial_state = comparator_reports_power_fail();

    power_fail_state = initial_state;
    power_candidate_state = initial_state;
    power_event_pending = initial_state ? 1U : 0U;

    power_confirm_ticks = 0;
    power_confirmation_pending = 0;

    ACSR = (1U << ACBG) | (1U << ACI);
    ACSR = (1U << ACBG) | (1U << ACIE);
}
```

Listing 3: Power-monitor initialization.

6.1 Pin direction and pull-up

The power-sense pin is configured as an input and its pull-up is disabled:

```
POWER_SENSE_DDR  &= ~(1U << POWER_SENSE_PIN);
POWER_SENSE_PORT &= ~(1U << POWER_SENSE_PIN);
```

The external divider already defines the voltage. Enabling the internal pull-up would alter the divider ratio and move the detection threshold.

6.2 Bandgap stabilization

The comparator is first enabled with its interrupt disabled:

```
ACSR = (1U << ACBG);
delay_ms(1);
```

The delay allows the internal reference and simulated comparator model to settle before the initial state is read. A 1 ms delay is intentionally conservative relative to the hardware startup requirement.

6.3 Failure already present at startup

After the delay, the raw state is copied into both the accepted state and candidate state. If it is already failed, `power_event_pending` is set immediately:

```
power_event_pending = initial_state ? 1U : 0U;
```

This is essential because a comparator interrupt is generated by a transition. If the voltage was already low at reset, no new edge may occur. The startup event ensures that the main loop still sends and logs the failure and enters the correct standby mode.

7 Comparator Interrupt Service Routine

```
ISR(ANA_COMP_vect)
{
    power_candidate_state = comparator_reports_power_fail();
    power_confirm_ticks = POWER_CONFIRM_TICKS_10MS;
    power_confirmation_pending = 1;
}
```

Listing 4: Comparator ISR.

The ISR is intentionally short. It does not send USART text, write EEPROM, update the LCD, or enter sleep. It only captures the raw state and restarts the confirmation interval. Heavy operations inside an ISR would increase latency, complicate nested timing, and risk blocking other interrupt-driven functions.

With `ACIS1:ACIS0=00`, the interrupt occurs when `ACO` toggles:

$$0 \to 1 \quad \text{or} \quad 1 \to 0.$$

A stable normal voltage does not repeatedly invoke the ISR. A stable failed voltage also does not repeatedly invoke it. Only threshold crossings or chatter create interrupts.

8 Software Stability Filter

8.1 Confirmation time constant

The project constant is

```
#define POWER_CONFIRM_TICKS_10MS 10U
```

Timer2 calls `power_monitor_tick_10ms()` approximately every 10 ms. Ten ticks correspond to approximately 100 ms:

$$10 \times 9.984 \text{ ms} \approx 99.84 \text{ ms}.$$

This rejects short switching transients and manually induced noise without making the system feel slow.

8.2 Timer2-side countdown

```
void power_monitor_tick_10ms(void)
{
    if (power_confirmation_pending && power_confirm_ticks > 0)
        power_confirm_ticks--;
}
```

The function is designed to be called from the Timer2 compare ISR. It performs only a bounded decrement. When the counter reaches zero, the candidate becomes eligible for main-loop validation.

8.3 Main-loop acceptance

```
void power_monitor_process(void)
{
    uint8_t candidate;
    uint8_t sreg;

    sreg = SREG;
    cli();
```

```
8
9       if (!power_confirmation_pending || power_confirm_ticks != 0) {
10          SREG = sreg;
11          return;
12      }
13
14      candidate = power_candidate_state;
15      power_confirmation_pending = 0;
16
17      SREG = sreg;
18
19      if (comparator_reports_power_fail() != candidate)
20          return;
21
22      if (candidate != power_fail_state) {
23          power_fail_state = candidate;
24          power_event_pending = 1;
25      }
26  }
```

Listing 5: Candidate validation and acceptance.

The atomic section prevents the comparator ISR or Timer2 ISR from changing the candidate and countdown while the main loop is deciding whether the interval has completed. Interrupts are restored to their previous state by writing back the saved `SREG`; the code does not assume that interrupts were enabled before entry.

After leaving the atomic section, the function reads the live comparator again. A candidate is rejected if the hardware no longer agrees. Finally, an event is produced only when the accepted state actually changes. This prevents duplicate events for the same stable condition.

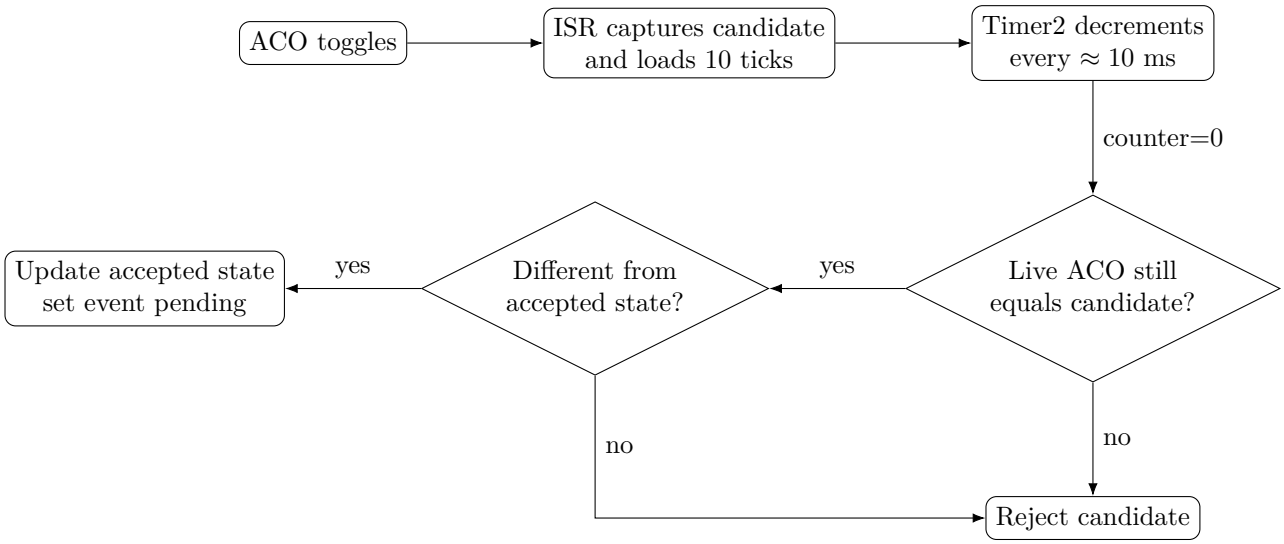

Figure 2: Temporal filtering and accepted-state transition logic.

9 Interaction with the Main Loop

The main loop repeatedly performs two operations:

```
1  power_monitor_process();
2  process_power_event();
```

The first accepts a stable candidate. The second, implemented in the application layer, consumes `power_event_pending`. On failure it typically:

1. clears the pending flag;
2. sends `POWER_FAIL` through USART;
3. waits for the final USART stop bit if necessary;
4. writes `EVT_POWER_FAIL` to the external EEPROM;
5. turns off or disables nonessential Master peripherals;
6. enters the dedicated restoration-wait state.

On restoration it sends `POWER RESTORED`, logs `EVT_POWER_OK`, reinitializes disabled peripherals, and returns to normal operation.

10 Normal Inactivity Sleep

Normal inactivity sleep uses Power-down mode and intentionally permits only INT0 to wake the Master. The comparator interrupt is therefore suspended before sleep.

10.1 Suspending comparator events

```
void power_monitor_suspend_for_sleep(void)
{
    uint8_t sreg = SREG;
    cli();

    power_confirm_ticks = 0;
    power_confirmation_pending = 0;

    ACSR = (1 << ACBG) | (1 << ACI);

    SREG = sreg;
}
```

The function cancels an unfinished candidate and clears any stale comparator interrupt flag. Because `ACIE` is omitted from the assignment, comparator interrupts are disabled. The comparator itself remains enabled because `ACBG` is retained and `ACD` remains zero.

10.2 Resynchronizing after INT0 wake

```
void power_monitor_resume_after_sleep(void)
{
    uint8_t current;
    uint8_t sreg;

    ACSR = (1 << ACBG) | (1 << ACI);
    ACSR = (1 << ACBG) | (1 << ACIE);

    current = comparator_reports_power_fail();

    sreg = SREG;
    cli();

    if (current != power_fail_state) {
        power_candidate_state = current;
```

```
        power_confirm_ticks = POWER_CONFIRM_TICKS_10MS;
        power_confirmation_pending = 1;
    } else {
        power_candidate_state = current;
        power_confirm_ticks = 0;
        power_confirmation_pending = 0;
    }

    SREG = sreg;
}
```

Power may have changed while the MCU was asleep, but that change was not allowed to wake it. After INT0 wakes the Master, the module reads the current comparator state. If it differs from the accepted state, the ordinary 100 ms confirmation process is restarted. No event is lost merely because the comparator interrupt was disabled during Power-down.

11 Power-Failure Standby and Restoration

The confirmed power-failure state is different from normal inactivity sleep. The Master has already sent and logged the failure. It disables nonessential peripherals, but the comparator must remain capable of detecting restoration.

11.1 Preparing for restoration

```
void power_monitor_prepare_restore_wait(void)
{
    uint8_t sreg = SREG;
    cli();

    power_candidate_state = 1U;
    power_confirm_ticks = 0;
    power_confirmation_pending = 0;

    ACSR = (1 << ACBG) | (1 << ACI);
    ACSR = (1 << ACBG) | (1 << ACIE);

    SREG = sreg;
}
```

The failure has already been accepted, so the filter is cleared. The comparator interrupt is re-enabled in toggle mode. In the surrounding standby code, other interrupt sources are disabled, and the MCU uses a sleep mode from which the comparator can wake it.

11.2 Raw state check

```
uint8_t power_monitor_raw_failed(void)
{
    return comparator_reports_power_fail();
}
```

This function is used by the standby loop to decide whether the failed condition still exists before sleeping again. It intentionally bypasses the normal 100 ms filter because Timer2 is stopped in the reduced-power state.

11.3 Accepting restoration

```
uint8_t power_monitor_accept_restore_if_present(void)
{
    uint8_t restored = 0;
    uint8_t sreg = SREG;

    cli();

    if (!comparator_reports_power_fail()) {
        power_fail_state = 0;
        power_event_pending = 1;
        power_candidate_state = 0;
        power_confirm_ticks = 0;
        power_confirmation_pending = 0;
        restored = 1;
    }

    SREG = sreg;
    return restored;
}
```

Restoration is accepted atomically when the raw comparator reports normal power. The accepted state is changed to zero and an event is queued for the application. The return value allows the standby loop to break and reinitialize the runtime peripherals.

This restoration path is intentionally immediate rather than temporally filtered because the 10 ms Timer2 time base is stopped during standby. The trade-off is greater sensitivity to chatter during restoration. In a physical design, hardware hysteresis or a short dedicated confirmation method can be added if required.

12 Two Sleep Policies

Table 3: Comparison of the two Master sleep conditions.

Characteristic	Normal inactivity sleep	Confirmed power-failure standby
Purpose	Save power after about 10 seconds without activity	Simulate a Master that shuts down most functions after supply failure
Typical mode	Power-down	Idle or another comparator-wake-capable state used by the application
Allowed wake source	INT0 only	Analog comparator restoration transition
Comparator interrupt	Disabled before sleep	Enabled
Timer2	Stopped by Power-down	Explicitly stopped by standby setup
Power change while asleep	Detected only after INT0 wake and resynchronization	Restoration wakes the processor

13 Concurrency and Atomicity

The module shares 8-bit variables among the main loop and two ISRs. Although individual 8-bit reads and writes are atomic on an 8-bit AVR, multi-step decisions are not. For example, the main loop must not observe `power_confirmation_pending=1`, then be interrupted while checking `power_confirm_ticks`, and continue with a different candidate. Saving `SREG`, executing `cli()`, and restoring `SREG` creates a short consistent snapshot without unconditionally enabling interrupts.

The ISR itself remains minimal, while slow operations such as EEPROM writes and USART transmission remain in the main context. This division is central to reliable embedded design.

14 Expected Operating Sequences

14.1 Normal-to-failed transition

1. V_{MON} falls until $V_{AIN1} < V_{BG}$.
2. `ACO` changes from zero to one.
3. The comparator ISR captures candidate state one and loads ten confirmation ticks.
4. Timer2 decrements the counter for approximately 100 ms.
5. The main loop verifies that the live comparator is still failed.
6. The accepted state changes from zero to one and `power_event_pending` becomes one.
7. The application sends the remote message, logs the event, disables peripherals, and enters restoration standby.

14.2 Failed-to-normal transition

1. V_{MON} rises until $V_{AIN1} > V_{BG}$.
2. `ACO` toggles to zero and wakes the MCU from power-failure standby.
3. The standby loop confirms that the raw failed condition is no longer present.
4. `power_monitor_accept_restore_if_present()` changes the accepted state to normal and queues an event.
5. Runtime peripherals are reinitialized.
6. The application sends and logs the restoration event.

14.3 Reset while voltage is already low

1. The MCU resets and calls `power_monitor_init()`.
2. The bandgap is enabled and allowed to settle for 1 ms.
3. The initial comparator state is read as failed.
4. `power_event_pending` is set even though no transition occurred.
5. The application immediately processes the existing failure and returns to standby.

15 Diagnostic Checks in Proteus

Useful logic and voltage probes are:

- a voltmeter at the potentiometer output V_{MON};
- a voltmeter at `PB3/AIN1`;
- a logic probe for a temporary application debug pin when an accepted event is processed;
- the Virtual Terminal to observe `POWER_FAIL` and `POWER RESTORED`;
- a watch on `ACO`, `power_candidate_state`, `power_confirm_ticks`, and `power_fail_state`.

At a normal monitored voltage of 5 V, the intended divider should produce approximately 1.56 V on AIN1. At the nominal trip point, AIN1 should be close to the comparator reference. If the observed monitored-voltage trip point is very different, both the source voltage and divider midpoint must be measured before changing resistor values.

16 Design Strengths and Limitations

16.1 Strengths

- The comparator ISR is short and deterministic.
- Startup failure is handled without requiring an edge.
- Temporal filtering rejects short disturbances.
- Accepted-state tracking prevents repeated duplicate events.
- Sleep integration preserves the requirement that INT0 is the only normal Power-down wake source.
- Complete `ACSR` assignments correctly handle the write-one-to-clear `ACI` flag.

16.2 Limitations

- The comparator threshold depends on bandgap and resistor tolerances.
- There is no hardware hysteresis; continuous noise near the threshold can repeatedly restart confirmation.
- Restoration during power-failure standby is accepted without the 100 ms filter because Timer2 is stopped.
- The simulation keeps the MCU supply alive while varying a separate monitored voltage; it does not model a true capacitor-backed last-gasp shutdown.

17 Complete Module Listing

The following listing is the supplied power module used as the basis of this report.

```
#include <avr/io.h>
#include <avr/interrupt.h>
#include "config.h"
#include "power.h"
#include "timer.h"

volatile uint8_t power_event_pending = 0;
volatile uint8_t power_fail_state = 0;

static volatile uint8_t power_candidate_state = 0;
static volatile uint8_t power_confirm_ticks = 0;
```

```
static volatile uint8_t power_confirmation_pending = 0;

static uint8_t comparator_reports_power_fail(void)
{
    return (ACSR & (1 << ACO)) ? 1U : 0U;
}

ISR(ANA_COMP_vect)
{
    power_candidate_state = comparator_reports_power_fail();
    power_confirm_ticks = POWER_CONFIRM_TICKS_10MS;
    power_confirmation_pending = 1;
}

void power_monitor_tick_10ms(void)
{
    if (power_confirmation_pending && power_confirm_ticks > 0)
        power_confirm_ticks--;
}

void power_monitor_process(void)
{
    uint8_t candidate;
    uint8_t sreg;

    sreg = SREG;
    cli();

    if (!power_confirmation_pending || power_confirm_ticks != 0) {
        SREG = sreg;
        return;
    }

    candidate = power_candidate_state;
    power_confirmation_pending = 0;

    SREG = sreg;

    if (comparator_reports_power_fail() != candidate)
        return;

    if (candidate != power_fail_state) {
        power_fail_state = candidate;
        power_event_pending = 1;
    }
}

void power_monitor_suspend_for_sleep(void)
{
    uint8_t sreg = SREG;
    cli();

    power_confirm_ticks = 0;
    power_confirmation_pending = 0;

    ACSR = (1 << ACBG) | (1 << ACI);

    SREG = sreg;
}
```

```
71
72  void power_monitor_resume_after_sleep(void)
73  {
74      uint8_t current;
75      uint8_t sreg;
76
77      ACSR = (1 << ACBG) | (1 << ACI);
78      ACSR = (1 << ACBG) | (1 << ACIE);
79
80      current = comparator_reports_power_fail();
81
82      sreg = SREG;
83      cli();
84
85      if (current != power_fail_state) {
86          power_candidate_state = current;
87          power_confirm_ticks = POWER_CONFIRM_TICKS_10MS;
88          power_confirmation_pending = 1;
89      } else {
90          power_candidate_state = current;
91          power_confirm_ticks = 0;
92          power_confirmation_pending = 0;
93      }
94
95      SREG = sreg;
96  }
97
98  void power_monitor_prepare_restore_wait(void)
99  {
100     uint8_t sreg = SREG;
101     cli();
102
103     power_candidate_state = 1U;
104     power_confirm_ticks = 0;
105     power_confirmation_pending = 0;
106
107     ACSR = (1 << ACBG) | (1 << ACI);
108     ACSR = (1 << ACBG) | (1 << ACIE);
109
110     SREG = sreg;
111 }
112
113 uint8_t power_monitor_raw_failed(void)
114 {
115     return comparator_reports_power_fail();
116 }
117
118 uint8_t power_monitor_accept_restore_if_present(void)
119 {
120     uint8_t restored = 0;
121     uint8_t sreg = SREG;
122
123     cli();
124
125     if (!comparator_reports_power_fail()) {
126         power_fail_state = 0;
127         power_event_pending = 1;
128         power_candidate_state = 0;
129         power_confirm_ticks = 0;
```

```
        power_confirmation_pending = 0;
        restored = 1;
    }

    SREG = sreg;
    return restored;
}

void power_monitor_init(void)
{
    uint8_t initial_state;

    POWER_SENSE_DDR &= ~(1U << POWER_SENSE_PIN);
    POWER_SENSE_PORT &= ~(1U << POWER_SENSE_PIN);

#ifdef ACME
    SFIOR &= ~(1U << ACME);
#endif

    ACSR = (1U << ACBG);
    delay_ms(1);

    initial_state = comparator_reports_power_fail();

    power_fail_state = initial_state;
    power_candidate_state = initial_state;
    power_event_pending = initial_state ? 1U : 0U;

    power_confirm_ticks = 0;
    power_confirmation_pending = 0;

    ACSR = (1U << ACBG) | (1U << ACI);
    ACSR = (1U << ACBG) | (1U << ACIE);
}
```

Listing 6: Complete power.c implementation.

18 Conclusion

The power module is a small event-driven state machine built on the ATmega32 analog comparator. Hardware detects threshold crossings immediately; a 100 ms software filter converts noisy raw transitions into stable application events. The design also distinguishes two power-management contexts: comparator-disabled normal Power-down sleep and comparator-enabled restoration standby. Startup synchronization, write-one-to-clear flag handling, atomic state updates, and separation of ISR work from main-loop work are the key implementation details that make the module dependable.

References

[1] Microchip Technology, *ATmega32/L Datasheet*, Analog Comparator, I/O Ports, Interrupts, and Sleep Modes.
https://ww1.microchip.com/downloads/en/DeviceDoc/doc2503.pdf

[2] SmartSafe Microprocessors End-Term Project Specification, Spring 1404.